Documentação do Sistema

Assistente de Receitas com IA (Google Gemini)

Matheus Figueira Merlos e João Pedro Marucci Pagluiso · Disciplina de Inteligência Artificial

1. Visão geral

O Assistente de Receitas recebe os ingredientes que o usuário tem em casa e devolve uma receita completa criada pela IA do Google Gemini. O objetivo é responder à pergunta "o que eu cozinho com o que já tenho?" e reduzir o desperdício de alimentos.

O sistema tem duas partes: uma interface web em React Native Web e uma API própria em Python (Flask), organizada no padrão MVC (Model, View e Controller). A API é a única parte que conversa com o Gemini: ela valida os dados, aplica as regras de negócio, monta o prompt, chama a IA e devolve a receita em um formato fixo.

2. Funcionalidades

- Geração de receita completa: título, descrição, tempo de preparo, rendimento, dificuldade, ingredientes, opcionais, passo a passo e dica do chef.
- Quatro tipos de refeição: Almoço/Jantar, Café da Manhã, Lanche e Sobremesa.
- Recusa de entradas que não são comida: formato inválido gera erro 400 e itens que não são alimentos (ex.: "cadeira") geram erro 422.
- A lista principal da receita só contém o que o usuário informou; extras como sal, açúcar e canela aparecem como opcionais.
- Se os ingredientes não combinam com o tipo escolhido, a IA cria o prato mais coerente e explica isso na descrição.
- Modelo reserva: se o modelo principal do Gemini falhar, um segundo modelo assume automaticamente.
- Mensagens de erro claras, separando cota esgotada (429) de Gemini sobrecarregado (503).
- Receitas salvas no navegador (localStorage), com opções de abrir, excluir e copiar o texto da receita.
- Indicador de status do servidor no topo da tela: online, offline ou online sem chave do Gemini.

3. Tecnologias utilizadas

Camada	Tecnologia	Função no projeto
Backend	Python 3 + Flask 3	Servidor HTTP e definição das rotas da API.
Backend	Flask-CORS 4	Libera o acesso do frontend (porta 5173) à API (porta 3001).
Backend	python-dotenv 1	Lê a `GEMINI_API_KEY` e a `PORT` do arquivo `.env`.
Backend	google-genai 2	SDK oficial do Google para chamar os modelos Gemini.
Backend	re, json, unicodedata, os	Bibliotecas nativas: validação por expressão regular, leitura do JSON, remoção de acentos e variáveis de ambiente.
Frontend	React 18 + React Native Web 0.19	Interface feita com componentes React Native (View, Text, TextInput, TouchableOpacity) renderizados no navegador.

Camada	Tecnologia	Função no projeto
Frontend	Vite 5	Servidor de desenvolvimento (porta 5173) e build; o alias `react-native → react-native-web` fica no `vite.config.js`.
Frontend	Fontes Bebas Neue e Caviar Dreams	Tipografia de títulos e textos.
IA	gemini-3.8-flash	Modelo principal.
IA	gemini-3.1-flash-lite	Modelo reserva (mais leve), usado quando o principal falha.

4. Arquitetura (MVC)

O sistema segue o modelo cliente-servidor, e o backend é organizado no padrão **MVC (Model-View-Controller)**. O frontend nunca chama o Gemini diretamente: isso protege a chave de API, permite validar antes de gastar cota e mantém o prompt sob controle do servidor.

```
[ Frontend: React Native Web (localhost:5173) ]
                |  POST /api/receitas/gerar  (JSON via fetch)
                v
[ Controller: receita_controller.py ]  ->  [ View: resposta_view.py ] -> JSON
                |
                v
[ Model: receita_model.py ]  ->  validação, prompt, trava de extras
                |  generate_content (SDK google-genai, resposta em JSON)
                v
[ Google Gemini: gemini-3.8-flash  |  reserva: gemini-3.1-flash-lite ]
```

Camada	Arquivo	Responsabilidade
Model	models/receita_model.py	Dados e regras: valida os ingredientes, monta o prompt, chama o Gemini com modelo reserva e separa os extras.
View	views/resposta_view.py	Monta as respostas em JSON: sucesso, erro, status e informações do serviço.
Controller	controllers/receita_controller.py	Recebe as requisições, chama o Model e escolhe a View e o código HTTP.
Inicialização	app.py	Cria o servidor Flask, liga o CORS e registra as rotas do Controller.
Interface	frontend/ (React Native Web)	Tela que o usuário vê: coleta os dados e mostra o JSON montado pela View.

A API é **stateless**: não há banco de dados nem sessão. Cada requisição é independente, e as receitas salvas ficam só no `localStorage` do navegador.

5. Estrutura de arquivos

```
Projeto IA/
├── backend/
│   ├── app.py                            cria o servidor e registra as rotas
│   ├── models/receita_model.py           Model: regras, prompt e Gemini
│   ├── views/resposta_view.py            View: respostas em JSON
│   ├── controllers/receita_controller.py  Controller: rotas e fluxo
│   ├── requirements.txt     flask, flask-cors, python-dotenv, google-genai
│   └── .env                 GEMINI_API_KEY e PORT (não versionar)
├── frontend/
│   ├── index.html           HTML e fontes
│   ├── vite.config.js       alias react-native -> react-native-web, porta 5173
│   ├── package.json
│   └── src/
│       ├── main.jsx         ponto de entrada do React
│       ├── App.jsx          estado global, abas, erros e receitas salvas
│       ├── services/api.js  checarStatus e requisitarReceita
│       └── components/
│           ├── Header.jsx           título, status do servidor e abas
│           ├── IngredientInput.jsx  ingredientes em chips e tipo de refeição
│           ├── RecipeCard.jsx       receita: salvar, copiar, voltar
│           └── SavedRecipes.jsx     receitas salvas: abrir e excluir
├── README.md
├── Apresentacao_Backend_Assistente_Receitas.pptx
└── Mostrando Sistema Funcionando.mp4
```

6. Backend por dentro

6.1 app.py (inicialização)

- `load_dotenv()` carrega o `.env`, para que a chave nunca fique escrita no código.
- `CORS(app)` libera o navegador a chamar a API a partir de outra porta.
- `app.register_blueprint(receita_bp)` registra as rotas do Controller.
- A porta vem de `PORT` (padrão 3001).

6.2 Model (models/receita_model.py)

Define a chave, o cliente do Gemini (só criado se houver chave) e os dois modelos: `gemini-3.8-flash` e `gemini-3.1-flash-lite`.

Função	O que faz
validar_formato_ingredientes	Aceita lista ou texto separado por vírgulas, descarta itens vazios e recusa: mais de 20 itens, itens com mais de 40 caracteres, com menos de 2 letras ou com símbolos fora de letras, números, espaço e - . , / ( ).
montar_prompt	Monta o prompt em cinco partes com os ingredientes e o tipo de refeição.
chamar_modelo	Chama generate_content com response_mime_type="application/json" e converte o texto com json.loads. Sem chave, lança ValueError.
gerar_receita	Tenta o modelo principal; se falhar, o reserva. Se os dois falharem, devolve o tipo do erro: cota, sobrecarga ou outro.
verificar_erro_cota	Procura 429, quota, resource_exhausted, rate limit ou too many requests na mensagem do erro.
verificar_erro_sobrecarga	Procura 503, unavailable, high demand ou overloaded.

Função	O que faz
normalizar	Tira acentos e deixa o texto em minúsculas ("Pão" vira "pao").
separar_ingredientes_extras	Mantém em "ingredientes" só os itens que contêm algum radical (até 5 letras, sem o "s" final) dos ingredientes do usuário; o resto vai para "ingredientesOpcionais".

6.3 View (views/resposta_view.py)

Função	Resposta
inicio(modelos)	Nome do serviço, status online e os dois modelos.
status(tem_chave)	Status online e se a chave está configurada (sem revelá-la).
sucesso(dados)	{"sucesso": true, "dados": ...} com código 200.
erro(mensagem, codigo)	{"sucesso": false, "mensagem": ...} com o código HTTP recebido.

6.4 Controller (controllers/receita_controller.py): a rota principal passo a passo

1. Lê o corpo com `request.get_json(silent=True) or {}`; um JSON malformado vira dicionário vazio.
2. Aplica o padrão `tipoRefeicao = "Almoco ou Jantar"` quando não enviado.
3. Sem ingredientes → 400. Formato inválido (verificado no Model) → 400 listando os itens recusados.
4. Chama gerar_receita no Model, que monta o prompt e tenta o modelo principal e depois o reserva.
5. Se os dois falharem: 429 (cota), 503 (sobrecarga) ou 500 (outro motivo), com as mensagens do dicionário MENSAGENS_ERRO_IA.
6. Se a IA marcar itens que não são alimentos → 422 com a lista desses itens.
7. Remove o campo de controle `ingredientesValidos`, aplica `separar_ingredientes_extras` e devolve a View de sucesso (200).

A rota inteira fica dentro de um `try/except`: qualquer erro inesperado vira 500 com mensagem amigável, sem derrubar o servidor. Os avisos `[Aviso]`, `[Fallback]` e `[Erro]` são impressos com `flush=True` para aparecerem no terminal na hora.

7. Endpoints da API

Método e rota	Chama a IA?	O que faz
GET /	Não	Identifica o serviço e lista os dois modelos configurados.
GET /api/status	Não	Informa se o servidor está online e se a chave existe (temChaveConfigurada), sem revelar a chave. Usada pelo frontend ao abrir.
POST /api/receitas/gerar	Sim	Gera a receita a partir dos ingredientes.

Entrada do POST

Campo	Tipo	Obrigatório	Regras
ingredientes	lista de textos ou texto com vírgulas	Sim	Até 20 itens; cada um com até 40 caracteres, pelo menos 2 letras e sem símbolos estranhos; só alimentos.

Campo	Tipo	Obrigatório	Regras
tipoRefeicao	texto	Não	Padrão "Almoco ou Jantar". O app envia Almoco/Jantar, Cafe da Manha, Lanche ou Sobremesa.

```
{
  "ingredientes": ["frango", "arroz", "tomate"],
  "tipoRefeicao": "Almoco/Jantar"
}
```

Resposta de sucesso (200)

```
{
  "sucesso": true,
  "dados": {
    "titulo": "Arroz de Frango ao Tomate",
    "descricao": "...",
    "tempoPreparo": "25 minutos",
    "rendimento": "2 porções",
    "dificuldade": "Facil",
    "ingredientes": ["300 g de frango", "1 xícara de arroz", "2 tomates"],
    "ingredientesOpcionais": ["Sal a gosto", "Azeite para refogar"],
    "modoPreparo": ["...", "..."],
    "dica": "..."
  }
}
```

O conteúdo de `dados` é ilustrativo: cada chamada gera uma receita diferente, mas sempre com esses nove campos. Toda resposta de erro segue o formato `{"sucesso": false, "mensagem": "..."}`.

8. Regras de negócio

O que pode ser verificado de forma determinística fica no código Python, antes de gastar uma chamada de IA. O que exige julgamento (o item é comida? o prato faz sentido?) vira instrução no prompt.

#	Regra	Onde	Comportamento
RN01	Pelo menos um ingrediente	Código	Ausente ou vazio → 400 "Informe pelo menos um ingrediente." A IA nem é chamada.
RN02	Corpo inválido não derruba a API	Código	JSON malformado vira {} e cai na RN01 (400).
RN03	Chave obrigatória	Código	Sem GEMINI_API_KEY, a geração falha com 500.
RN04	Tipo de refeição padrão	Código	Não informado → "Almoco ou Jantar".
RN05	Lista ou texto	Código	"ovo, queijo" e ["ovo", "queijo"] viram a mesma lista; itens vazios são descartados.
RN06	Lista principal só com o que o usuário tem	Prompt + trava no código	A IA só lista o que foi informado; se incluir um extra (ex.: açúcar), separar_ingredientes_extras o move para os opcionais.
RN07	Extras só como opcionais	Prompt	Sal, açúcar, canela, leite etc. ficam em ingredientesOpcionais; a receita funciona sem eles e o preparo os marca como "(opcional)".
RN08	JSON com 9 campos fixos	Prompt + modo JSON	titulo, descricao, tempoPreparo, rendimento, dificuldade, ingredientes, ingredientesOpcionais, modoPreparo e dica.

#	Regra	Onde	Comportamento
RN09	Formato do ingrediente	Código	Até 20 itens, 40 caracteres, pelo menos 2 letras, sem símbolos; senão 400 listando os itens.
RN10	Só alimentos reais	Prompt	Se algum item não for comestível, a IA devolve ingredientesValidos: false e a API responde 422 listando os itens.
RN11	Tipo de refeição não é forçado	Prompt	Ingredientes que não combinam com o tipo (ex.: carne moída em sobremesa) geram o prato mais coerente, explicado na descrição.

O usuário não informa tempo: a IA estima o tempo de preparo de cada receita.

9. Integração com o Gemini

9.1 Prompt em cinco partes

1. **Persona:** "chef de cozinha experiente, criativo e especialista em receitas práticas do dia a dia".
2. **Verificação prévia (RN10):** antes de tudo, a IA confere se todos os itens são alimentos; se algum não for, devolve só `{"ingredientesValidos": false, "itensInvalidos": [...]}`.
3. **Dados do usuário:** `{lista_ingredientes}` e `{tipo_refeicao}`.
4. **Diretrizes obrigatórias:** RN06, RN07, RN11 e passo a passo objetivo.
5. **Formato de saída:** molde JSON exato com `ingredientesValidos: true` e os nove campos da receita.

9.2 Garantia de resposta em JSON

Camada	Mecanismo	O que evita
1	response_mime_type="application/json" na chamada	O modelo responder com texto livre ou Markdown.
2	json.loads dentro de try	Um JSON inválido virar erro não tratado: ele aciona o modelo reserva.

Como o modo JSON já garante o formato, não há etapa de limpeza de marcações Markdown: a resposta vai direto para o json.loads.

9.3 Fallback e classificação de erros

Situação	Resposta
Principal responde certo	200 com a receita.
Principal falha, reserva responde	200 com a receita do reserva (o usuário nem percebe).
Os dois falham e algum erro é de cota	429 "A cota de uso do Gemini foi esgotada..." (a cota tem prioridade).
Os dois falham por sobrecarga	503 "O Gemini esta sobrecarregado no momento. Tente novamente em alguns segundos."
Os dois falham por outro motivo	500 "Nao foi possivel gerar a receita no momento..."

Acompanhar a cota: a API do Gemini não informa quanto de cota resta. O uso por modelo e os limites do plano (requisições por minuto e por dia) aparecem no Google AI Studio, na conta dona da chave.

10. Códigos de resposta

Código	Quando	Mensagem
200	Receita gerada	— (sucesso: true + dados)
400	Sem ingredientes, formato inválido ou corpo inválido	"Informe pelo menos um ingrediente." ou "Ingrediente(s) com formato invalido: ..."
422	A IA identificou itens que não são alimentos	"Alguns itens informados nao sao ingredientes culinarios: cadeira, celular. Remova-os e tente novamente."
429	Cota do Gemini esgotada	"A cota de uso do Gemini foi esgotada. Aguarde alguns minutos..."
503	Gemini sobrecarregado	"O Gemini esta sobrecarregado no momento. Tente novamente em alguns segundos."
500	Outra falha da IA ou erro inesperado	"Nao foi possivel gerar a receita no momento..." ou "Erro interno no servidor ao processar a receita."

11. Fluxo de execução

1. **Inicialização:** python app.py carrega o .env, cria o cliente Gemini e sobe na porta 3001.
2. **Abertura do app:** o frontend chama GET /api/status e mostra se o servidor está online e com chave.
3. **Entrada:** o usuário adiciona ingredientes como chips (Enter ou botão Adicionar; duplicados são avisados), escolhe o tipo de refeição e clica em Criar Receita.
4. **Requisição:** requisitarReceita() envia POST /api/receitas/gerar com o JSON.
5. **Validação:** ingredientes ausentes ou com formato inválido → 400 imediato.
6. **Prompt:** dados inseridos no molde com persona, verificação prévia, diretrizes e formato JSON.
7. **IA:** modelo principal, ou reserva se falhar; item que não é alimento → 422.
8. **Trava:** extras não informados vão para os opcionais.
9. **Resposta:** {sucesso, dados} ou {sucesso: false, mensagem}.
10. **Exibição:** o RecipeCard mostra a receita; em erro, o App mostra a mensagem num alerta vermelho.

12. Frontend

Arquivo	Responsabilidade
App.jsx	Consulta o status do servidor ao abrir, guarda o estado (aba ativa, receita atual, carregando, mensagem de erro, receitas salvas), carrega e grava as salvas no localStorage (chave receitas_salvas) e alterna entre as abas.
Header.jsx	Título "Assistente de Receitas", aviso de status do servidor e abas Gerar Receita / Receitas Salvas.
IngredientInput.jsx	Campo de texto, chips removíveis (começa com Ovos, Tomate, Queijo), aviso de item duplicado e seleção do tipo de refeição.
RecipeCard.jsx	Mostra título, descrição, tempo, rendimento, dificuldade, ingredientes, "Opcionais (voce nao informou)", passo a passo e dica; botões Salvar/Salva, Copiar e Voltar para o formulario.
SavedRecipes.jsx	Lista as receitas salvas com Abrir e Excluir; mostra aviso quando não há nenhuma.

Arquivo	Responsabilidade
services/api.js	checarStatus() (GET /api/status, devolve offline se falhar) e requisitarReceita() (POST, lança erro com a mensagem da API).

13. Como executar

Pré-requisitos: Python 3, Node.js e uma chave do Gemini gerada no Google AI Studio.

Backend

```
cd backend
pip install -r requirements.txt
# criar backend/.env com GEMINI_API_KEY=sua_chave_aqui e PORT=3001
python app.py
```

Frontend (em outro terminal)

```
cd frontend
npm install
npm start
# abrir http://localhost:5173
```

Teste rápido da API sem o frontend

```
curl -X POST http://localhost:3001/api/receitas/gerar -H "Content-Type: application/json" \
     -d "{\"ingredientes\": [\"ovo\", \"queijo\"], \"tipoRefeicao\": \"Lanche\"}"
```

14. Limitações e melhorias futuras

- A trava de extras compara o começo das palavras: se o usuário digitar "frango" e a receita escrever "peito de ave", o item vai para os opcionais por engano.
- A verificação "é alimento?" depende da IA; casos ambíguos seguem o julgamento do Gemini.
- Validar tipoRefeicao contra a lista fixa de opções do frontend.
- Conferir se o JSON da IA tem os nove campos antes de devolver ao frontend.
- Guardar em cache receitas já geradas para os mesmos ingredientes, economizando cota.
- Restringir o CORS à origem do frontend em produção, trocar print por logging e criar testes automatizados (pytest com o Gemini simulado).